

AI ASSISTANT CODING ASSIGNMENT - 3.2

NAME : G.Vamshi

HT.NO : 2303A51448

BATCH : 21

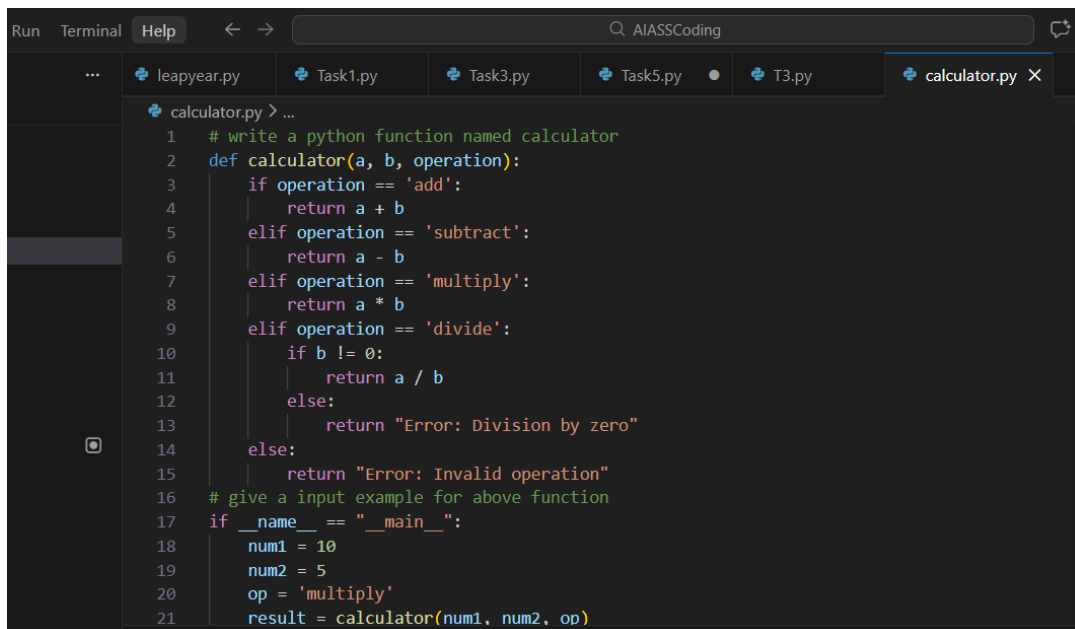
Task Description-1

- **Progressive Prompting for Calculator Design:** Ask the AI to design a simple calculator program by initially providing only the function name. Gradually enhance the prompt by adding comments and usage examples.

Expected Output-1

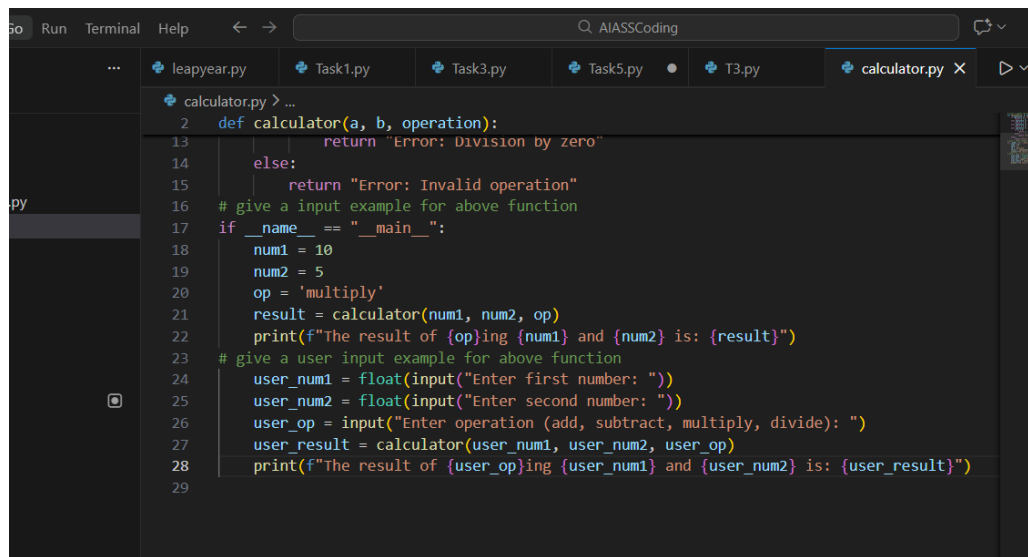
- **Comparison showing improvement in AI-generated calculator logic and structure.**
-

CODE :

A screenshot of a code editor window with a dark theme. The editor has a top bar with 'Run', 'Terminal', and 'Help' menus, and a search bar containing 'AIASSCoding'. Below the top bar is a tab bar with several open files: 'leapyear.py', 'Task1.py', 'Task3.py', 'Task5.py', 'T3.py', and 'calculator.py'. The 'calculator.py' file is selected and its content is displayed in the main editor area. The code is a Python function named 'calculator' that takes two numbers 'a' and 'b', and a string 'operation' as arguments. It uses if-elif-else statements to perform addition, subtraction, multiplication, and division. It also includes error handling for division by zero and invalid operations. At the bottom, there is a main block that initializes variables and calls the calculator function.

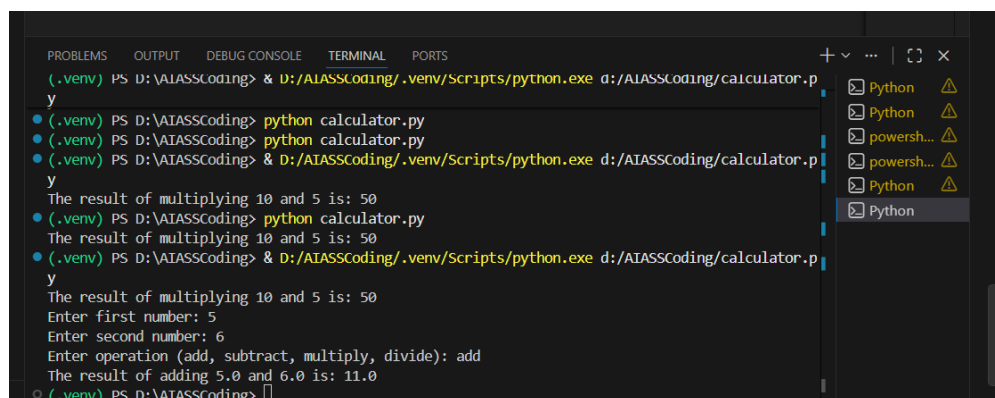
```
1 # write a python function named calculator
2 def calculator(a, b, operation):
3     if operation == 'add':
4         return a + b
5     elif operation == 'subtract':
6         return a - b
7     elif operation == 'multiply':
8         return a * b
9     elif operation == 'divide':
10        if b != 0:
11            return a / b
12        else:
13            return "Error: Division by zero"
14    else:
15        return "Error: Invalid operation"
16 # give a input example for above function
17 if __name__ == "__main__":
18     num1 = 10
19     num2 = 5
20     op = 'multiply'
21     result = calculator(num1, num2, op)
```

CODE AFTER ENHANCING THE PROMPT :



```
calculator.py > ...
2 def calculator(a, b, operation):
13     return "Error: Division by zero"
14 else:
15     return "Error: Invalid operation"
16 # give a input example for above function
17 if __name__ == "__main__":
18     num1 = 10
19     num2 = 5
20     op = 'multiply'
21     result = calculator(num1, num2, op)
22     print(f"The result of {op}ing {num1} and {num2} is: {result}")
23 # give a user input example for above function
24 user_num1 = float(input("Enter first number: "))
25 user_num2 = float(input("Enter second number: "))
26 user_op = input("Enter operation (add, subtract, multiply, divide): ")
27 user_result = calculator(user_num1, user_num2, user_op)
28 print(f"The result of {user_op}ing {user_num1} and {user_num2} is: {user_result}")
29
```

OUTPUT :



```
(.venv) PS D:\AIASSCoding> & D:/AIASSCoding/.venv/Scripts/python.exe d:/AIASSCoding/calculator.py
(.venv) PS D:\AIASSCoding> python calculator.py
(.venv) PS D:\AIASSCoding> python calculator.py
(.venv) PS D:\AIASSCoding> & D:/AIASSCoding/.venv/Scripts/python.exe d:/AIASSCoding/calculator.py
The result of multiplying 10 and 5 is: 50
(.venv) PS D:\AIASSCoding> python calculator.py
The result of multiplying 10 and 5 is: 50
(.venv) PS D:\AIASSCoding> & D:/AIASSCoding/.venv/Scripts/python.exe d:/AIASSCoding/calculator.py
The result of multiplying 10 and 5 is: 50
Enter first number: 5
Enter second number: 6
Enter operation (add, subtract, multiply, divide): add
The result of adding 5.0 and 6.0 is: 11.0
(.venv) PS D:\AIASSCoding>
```

Final Observation :

At first, when only the function name was given, the AI generated a very basic and incomplete calculator function with little or no logic. After adding comments, the AI started including parameters and arithmetic operations. When usage examples were finally added, the AI produced a complete and well-structured calculator program with proper conditions and error handling. This clearly shows that progressive prompting improves both the logic and structure of the generated code.

Task Description-2

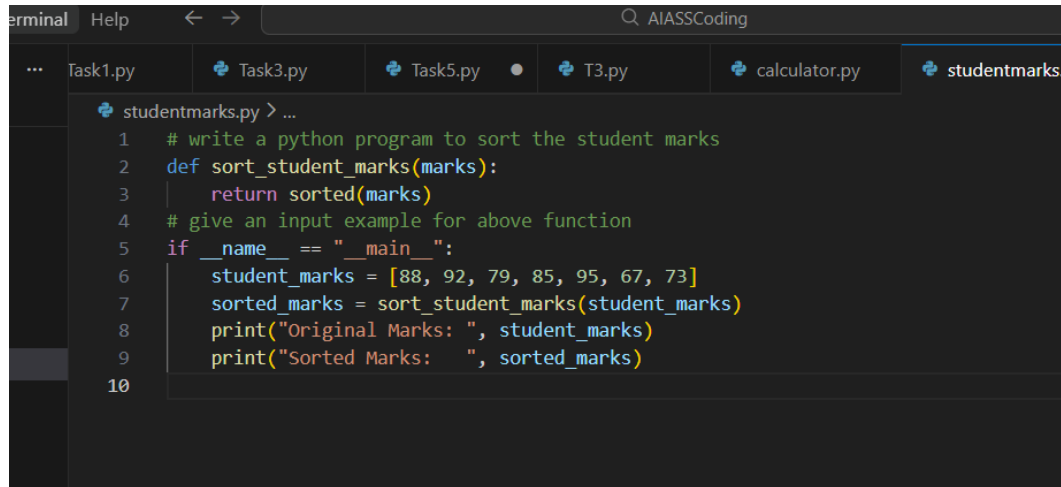
- **Refining Prompts for Sorting Logic:** Start with a vague prompt for sorting student marks, then refine it to clearly specify sorting order and constraints.

Expected Output-2

- AI-generated sorting function evolves from ambiguous logic to an accurate and efficient Implementation.
-

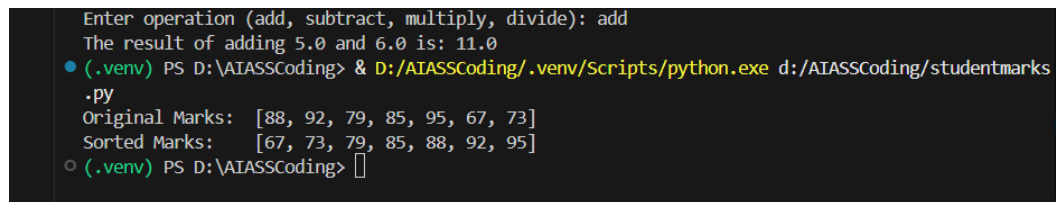
STAGE 1 :

Code :



```
terminal Help < > AIASSCoding
Task1.py Task3.py Task5.py T3.py calculator.py studentmarks.py
studentmarks.py > ...
1 # write a python program to sort the student marks
2 def sort_student_marks(marks):
3     return sorted(marks)
4 # give an input example for above function
5 if __name__ == "__main__":
6     student_marks = [88, 92, 79, 85, 95, 67, 73]
7     sorted_marks = sort_student_marks(student_marks)
8     print("Original Marks: ", student_marks)
9     print("Sorted Marks: ", sorted_marks)
10
```

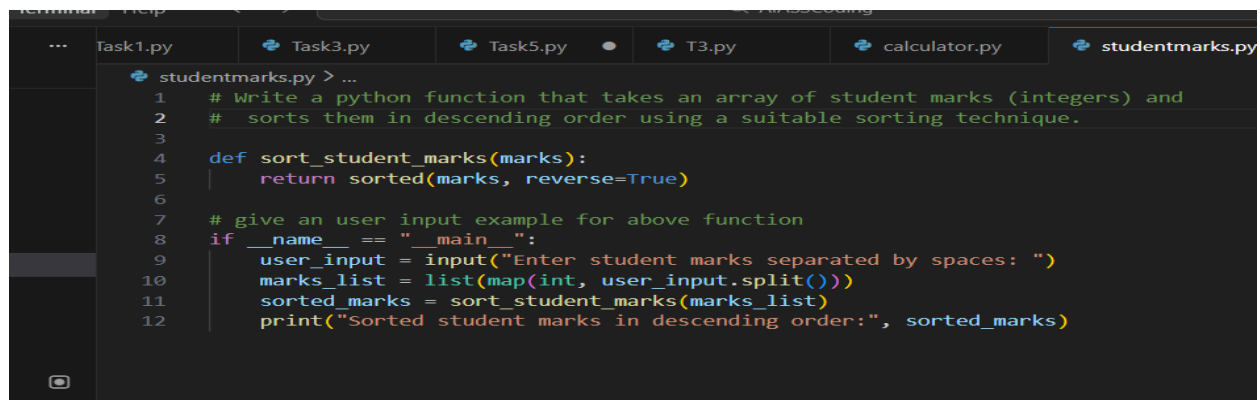
Output :



```
Enter operation (add, subtract, multiply, divide): add
The result of adding 5.0 and 6.0 is: 11.0
(.venv) PS D:\AIASSCoding> & D:\AIASSCoding\.venv\Scripts\python.exe d:\AIASSCoding\studentmarks.py
Original Marks: [88, 92, 79, 85, 95, 67, 73]
Sorted Marks: [67, 73, 79, 85, 88, 92, 95]
(.venv) PS D:\AIASSCoding>
```

STAGE 2 :

Code :



```
terminal Help < > AIASSCoding
Task1.py Task3.py Task5.py T3.py calculator.py studentmarks.py
studentmarks.py > ...
1 # Write a python function that takes an array of student marks (integers) and
2 # sorts them in descending order using a suitable sorting technique.
3
4 def sort_student_marks(marks):
5     return sorted(marks, reverse=True)
6
7 # give an user input example for above function
8 if __name__ == "__main__":
9     user_input = input("Enter student marks separated by spaces: ")
10    marks_list = list(map(int, user_input.split()))
11    sorted_marks = sort_student_marks(marks_list)
12    print("Sorted student marks in descending order:", sorted_marks)
```

Output :

```

(.venv) PS D:\AIASSCoding> & D:/AIASSCoding/.venv/Scripts/python.exe d:/AIASSCoding/studentmarks
.py
Enter student marks separated by spaces: 57 63 45 78 90 46
Sorted student marks in descending order: [90, 78, 63, 57, 46, 45]
(.venv) PS D:\AIASSCoding>

```

STAGE 3 :

Code :

```

sk1.py Task3.py Task5.py T3.py calculator.py studentmarks.py X
studentmarks.py > ...
1 # Write an optimized python function to sort an array of student marks in descending order
2 # Constraints:
3 # - Marks range from 0 to 100
4 # - Do not use built-in sort functions
5 # - Time complexity should be efficient
6 # - Handle duplicate marks properly
7 def sort_student_marks(marks):
8     # Counting sort is efficient for small integer ranges like 0-100
9     count = [0] * 101 # Initialize count array for marks 0-100
10    for mark in marks:
11        count[mark] += 1
12
13    sorted_marks = []
14    for i in range(100, -1, -1): # Traverse from 100 down to 0
15        sorted_marks.extend([i] * count[i])
16
17    return sorted_marks
18 # user input example for the function
19 if __name__ == "__main__":
20     user_input = input("Enter student marks separated by spaces: ")
21     marks_list = list(map(int, user_input.split()))
22     sorted_marks = sort_student_marks(marks_list)
23     print("Sorted student marks in descending order:")
24     print(sorted_marks)

```

Output :

```

(.venv) PS D:\AIASSCoding> & D:/AIASSCoding/.venv/Scripts/python.exe d:/AIASSCoding/studentmarks
.py
Enter student marks separated by spaces: 40 60 78 90 45 23
Sorted student marks in descending order:
[90, 78, 60, 45, 40, 23]
(.venv) PS D:\AIASSCoding>

```

Final Observation :

the AI produced a simple sorting solution without clear direction or constraints. After refining the prompt to specify sorting order, the output became more accurate and meaningful. When clear constraints and examples were added, the AI generated a more structured and efficient sorting function. This demonstrates that refining prompts helps the AI move from ambiguous logic to a correct and reliable implementation.

Task Description-3

- **Few-Shot Prompting for Prime Number Validation:** Provide multiple input-output

examples for a function that checks whether a number is prime. Observe how few-shot

prompting improves correctness.

Expected Output-3

- Improved prime-checking function with better edge-case handling.
-

STAGE 1 :

Code :

```
prime.py > ...
1  # write a python code to check if a number is prime
2  def is_prime(n):
3      """Check if a number is prime."""
4      if n <= 1:
5          return False
6      for i in range(2, int(n**0.5) + 1):
7          if n % i == 0:
8              return False
9      return True
10 # example usage with user input
11 if __name__ == "__main__":
12     user_input = int(input("Enter a number to check if it's prime: "))
13     if is_prime(user_input):
14         print(f"{user_input} is a prime number.")
15     else:
16         print(f"{user_input} is not a prime number.")
```

Output :

```
(.venv) PS D:\AIASSCoding> .\prime.py
(.venv) PS D:\AIASSCoding> & D:/AIASSCoding/.venv/Scripts/python.exe d:/AIASSCoding/prime.py
Enter a number to check if it's prime: 11
11 is a prime number.
(.venv) PS D:\AIASSCoding>
```

STAGE 2

Code :

```
prime.py > ...
1 # Write a Python function is_prime(n) that returns True if n is prime, else False.
2 # Examples:
3 # is_prime(2) → True
4 # is_prime(3) → True
5 # is_prime(4) → False
6 # is_prime(9) → False
7 # is_prime(11) → True
8 def is_prime(n):
9     """
10     Check if a number is prime.
11     A prime number is greater than 1 and has no divisors other than 1 and itself.
12     """
13     if n <= 1:
14         return False
15     if n <= 3:
16         return True
17     if n % 2 == 0 or n % 3 == 0:
18         return False
19
20     i = 5
21     while i * i <= n:
22         if n % i == 0 or n % (i + 2) == 0:
23             return False
24         i += 6
25     return True
26 # example usage
27 if __name__ == "__main__":
28     test_numbers = [2, 3, 4, 9, 11, 15, 17, 18, 19, 20]
```

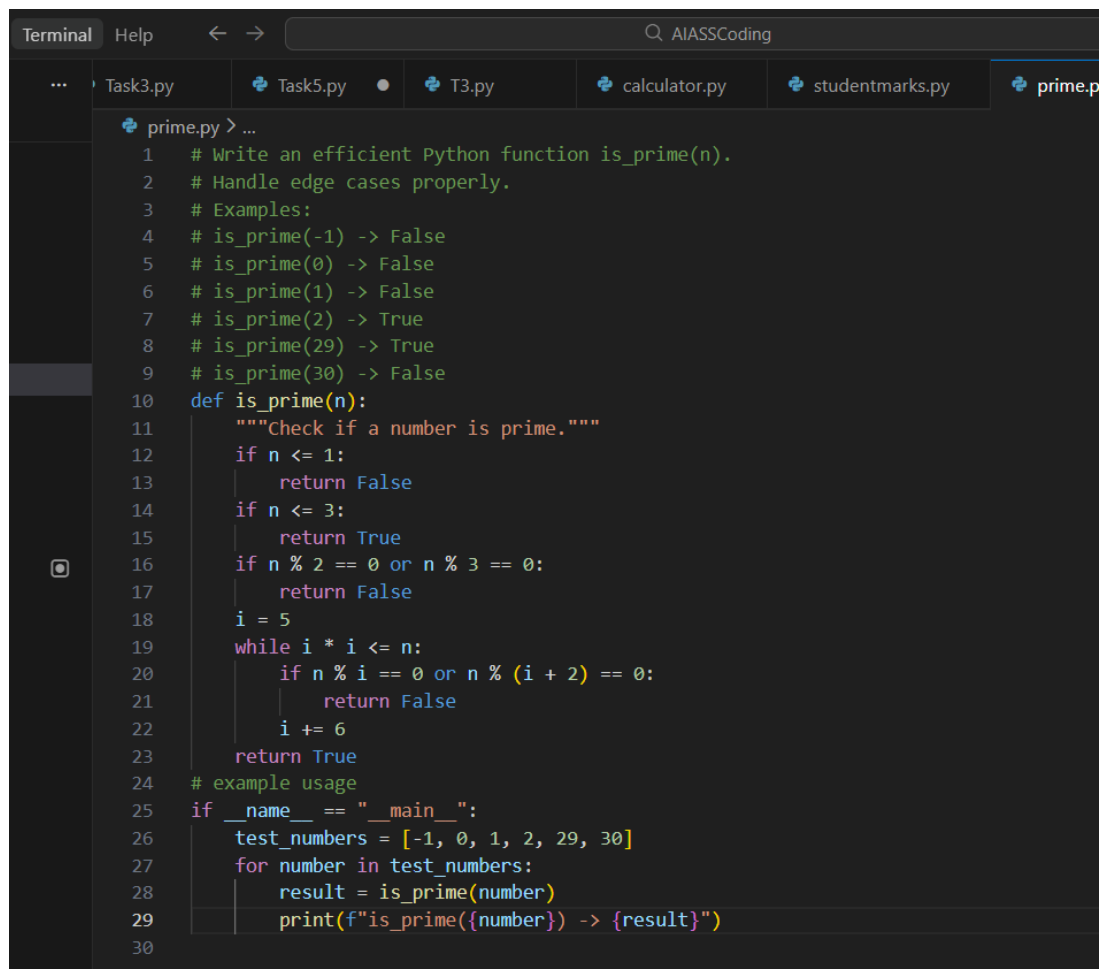
```
23         return False
24         i += 6
25     return True
26 # example usage
27 if __name__ == "__main__":
28     test_numbers = [2, 3, 4, 9, 11, 15, 17, 18, 19, 20]
29     for number in test_numbers:
30         result = is_prime(number)
31         print(f"is_prime({number}) → {result}")
```

Output :

```
(.venv) PS D:\AIASSCoding> & D:\AIASSCoding\.venv\Scripts\python.exe d:\AIASSCoding\prime.py
is_prime(11) → True
is_prime(15) → False
is_prime(17) → True
is_prime(18) → False
is_prime(19) → True
is_prime(20) → False
(.venv) PS D:\AIASSCoding>
```

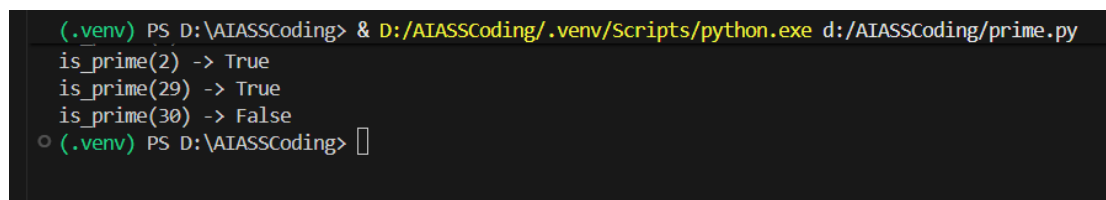
STAGE 3 :

Code :

A screenshot of a code editor window titled "AIASSCoding". The editor shows a Python file named "prime.py" with the following code:

```
1 # Write an efficient Python function is_prime(n).
2 # Handle edge cases properly.
3 # Examples:
4 # is_prime(-1) -> False
5 # is_prime(0) -> False
6 # is_prime(1) -> False
7 # is_prime(2) -> True
8 # is_prime(29) -> True
9 # is_prime(30) -> False
10 def is_prime(n):
11     """Check if a number is prime."""
12     if n <= 1:
13         return False
14     if n <= 3:
15         return True
16     if n % 2 == 0 or n % 3 == 0:
17         return False
18     i = 5
19     while i * i <= n:
20         if n % i == 0 or n % (i + 2) == 0:
21             return False
22         i += 6
23     return True
24 # example usage
25 if __name__ == "__main__":
26     test_numbers = [-1, 0, 1, 2, 29, 30]
27     for number in test_numbers:
28         result = is_prime(number)
29         print(f"is_prime({number}) -> {result}")
30
```

Output :

A screenshot of a terminal window showing the execution of the prime.py script. The prompt is "(.venv) PS D:\AIASSCoding>". The command executed is "& D:/AIASSCoding/.venv/Scripts/python.exe d:/AIASSCoding/prime.py". The output is:

```
is_prime(2) -> True
is_prime(29) -> True
is_prime(30) -> False
(.venv) PS D:\AIASSCoding>
```

Final Observation :

In the initial prompt without examples, the AI generated a basic prime-checking function that could miss important edge cases. When one example was provided, the result improved slightly. After giving multiple input-output examples (few-shot prompting), the AI clearly handled cases like 0, 1, and negative numbers and produced

a more accurate and robust prime-checking function. This shows that few-shot prompting improves correctness and edge-case handling.

Task Description-4

- **Prompt-Guided UI Design for Student Grading System:** Create a user interface for a

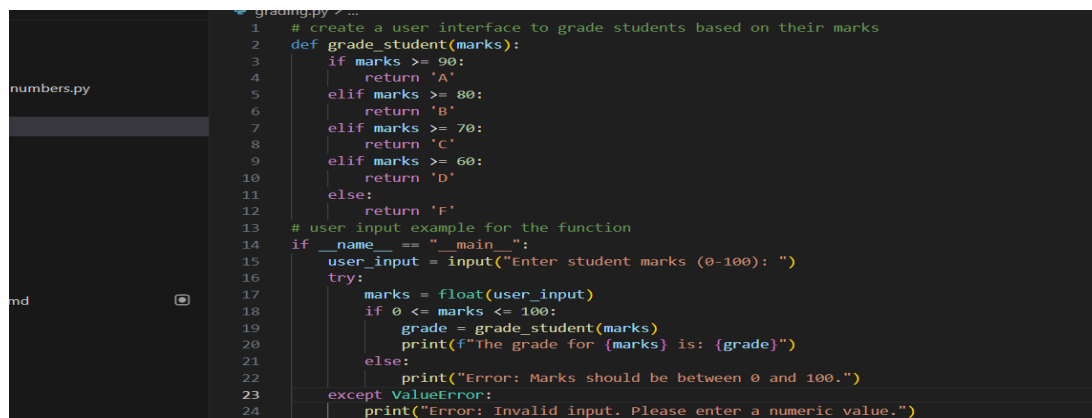
student grading system that calculates total marks, percentage, and grade based on user input.

Expected Output-4

- **Well-structured UI code with accurate calculations and clear output display.**
-

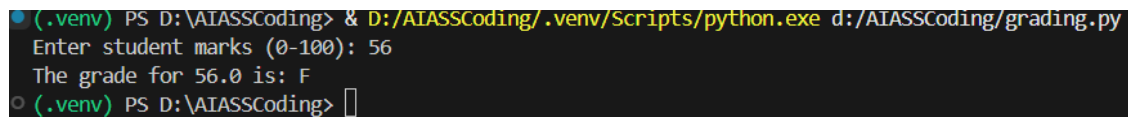
STAGE 1:

Code :



```
1 # create a user interface to grade students based on their marks
2 def grade_student(marks):
3     if marks >= 90:
4         return 'A'
5     elif marks >= 80:
6         return 'B'
7     elif marks >= 70:
8         return 'C'
9     elif marks >= 60:
10        return 'D'
11    else:
12        return 'F'
13
14 # user input example for the function
15 if __name__ == "__main__":
16     user_input = input("Enter student marks (0-100): ")
17     try:
18         marks = float(user_input)
19         if 0 <= marks <= 100:
20             grade = grade_student(marks)
21             print(f"The grade for {marks} is: {grade}")
22         else:
23             print("Error: Marks should be between 0 and 100.")
24     except ValueError:
25         print("Error: Invalid input. Please enter a numeric value.")
```

Output :



```
(.venv) PS D:\AIASSCoding> & D:\AIASSCoding\.venv\Scripts\python.exe d:/AIASSCoding/grading.py
Enter student marks (0-100): 56
The grade for 56.0 is: F
(.venv) PS D:\AIASSCoding>
```

STAGE 2 :

Code :

```

grading.py > ...
1 # Create a student grading system UI that accepts marks from the user,
2 # calculates total marks and percentage, and displays the result.
3 def calculate_grades(marks):
4     total_marks = sum(marks)
5     max_marks = len(marks) * 100 # Assuming each subject is out of 100
6     percentage = (total_marks / max_marks) * 100
7
8     if percentage >= 90:
9         grade = 'A'
10    elif percentage >= 80:
11        grade = 'B'
12    elif percentage >= 70:
13        grade = 'C'
14    elif percentage >= 60:
15        grade = 'D'
16    else:
17        grade = 'F'
18
19    return total_marks, percentage, grade
20 # example usage with user input
21 if __name__ == "__main__":
22     user_input = input("Enter student marks separated by spaces: ")
23     marks_list = list(map(int, user_input.split()))
24     total, percent, grade = calculate_grades(marks_list)
25     print(f"Total Marks: {total}")
26     print(f"Percentage: {percent:.2f}%")
27     print(f"Grade: {grade}")
28

```

Output :

```

(.venv) PS D:\AIASSCoding> & D:/AIASSCoding/.venv/scripts/python.exe d:/AIASSCoding/grading.py
Enter student marks separated by spaces: 23 45 67 89 90 12
Total Marks: 326
Percentage: 54.33%
Grade: F
(.venv) PS D:\AIASSCoding>

```

STAGE 3 :

Code :

```

D:\AIASSCoding\Task5.py
1 # Create a well-structured UI for a student grading system.
2 # Allow users to enter marks for 5 subjects.
3 # Calculate total marks, percentage, and assign grades
4 # (A: >=90, B: >=75, C: >=50, Fail: <50).
5 # Display all results clearly.
6 def calculate_grade(marks):
7     total_marks = sum(marks)
8     percentage = (total_marks / (len(marks) * 100)) * 100
9
10    if percentage >= 90:
11        grade = 'A'
12    elif percentage >= 75:
13        grade = 'B'
14    elif percentage >= 50:
15        grade = 'C'
16    else:
17        grade = 'Fail'
18
19    return total_marks, percentage, grade
20 # User input and display results
21 if __name__ == "__main__":
22     print("Student Grading System")
23     print("=====")
24
25     marks = []
26     for i in range(1, 6):
27         while True:

```

```

24     s
25     marks = []
26     for i in range(1, 6):
27         while True:
28             try:
29                 mark = float(input(f"Enter marks for subject {i} (0-100): "))
30                 if 0 <= mark <= 100:
31                     marks.append(mark)
32                     break
33             else:
34                 print("Please enter a valid mark between 0 and 100.")
35         except ValueError:
36             print("Invalid input. Please enter a numeric value.")
37
38     total, percentage, grade = calculate_grade(marks)
39
40     print("\nResults:")
41     print("-----")
42     print(f"Total Marks: {total} out of {len(marks) * 100}")
43     print(f"Percentage: {percentage:.2f}%")
44     print(f"Grade: {grade}")

```

Output :

```

(.venv) PS D:\AIASSCoding> & D:/AIASSCoding/.venv/Scripts/python.exe d:/AIASSCoding/grading.py
=====
Enter marks for subject 1 (0-100): 56
Enter marks for subject 2 (0-100): 78
Enter marks for subject 3 (0-100): 90
Enter marks for subject 4 (0-100): 56
Enter marks for subject 5 (0-100): 72

Results:
-----
Total Marks: 352.0 out of 500
Percentage: 70.40%
Grade: C
(.venv) PS D:\AIASSCoding>

```

Final Observation :

With a vague UI prompt, the AI produced only a simple or unclear interface idea. As the prompt was refined to include calculation requirements, the UI output became more meaningful. When full instructions were given (inputs, calculations, grade rules, and display), the AI generated a well-structured user interface with correct total, percentage, and grade calculation along with clear result display. This shows that prompt guidance greatly improves UI structure and usability.

Task Description-5

• Analyzing Prompt Specificity in Unit Conversion Functions: Improving a Unit

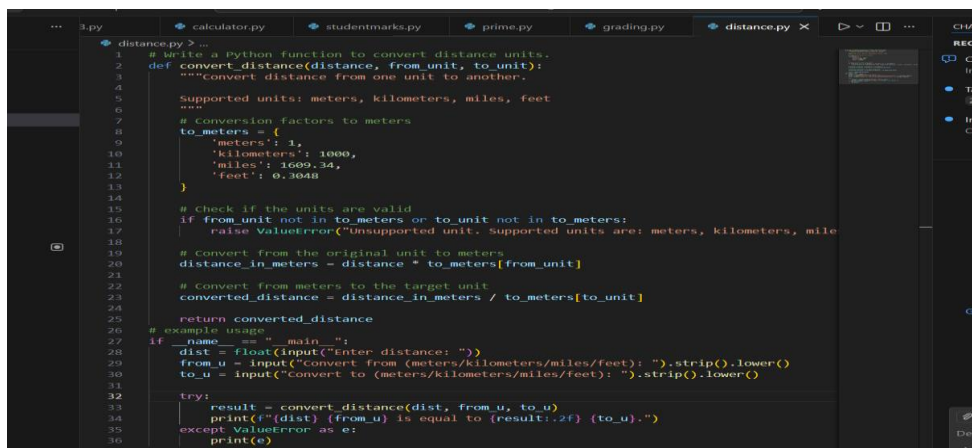
Conversion Function (Kilometers to Miles and Miles to Kilometers) Using Clear Instructions.

Expected Output-5

- Analysis of code quality and accuracy differences across multiple prompt variations.
-

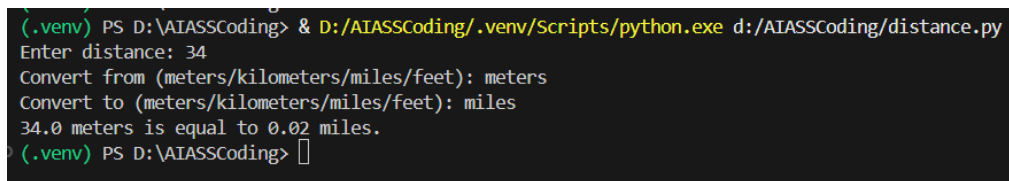
STAGE 1:

Code :



```
1 # write a python function to convert distance units.
2 def convert_distance(distance, from_unit, to_unit):
3     """Convert distance from one unit to another.
4
5     Supported units: meters, kilometers, miles, feet
6     """
7     # conversion factors to meters
8     to_meters = {
9         'meters': 1,
10        'kilometers': 1000,
11        'miles': 1609.34,
12        'feet': 0.3048
13    }
14
15    # Check if the units are valid
16    if from_unit not in to_meters or to_unit not in to_meters:
17        raise ValueError("Unsupported unit. Supported units are: meters, kilometers, mile
18
19    # Convert from the original unit to meters
20    distance_in_meters = distance * to_meters[from_unit]
21
22    # Convert from meters to the target unit
23    converted_distance = distance_in_meters / to_meters[to_unit]
24
25    return converted_distance
26
27 # example usage
28 if __name__ == "__main__":
29     dist = float(input("Enter distance: "))
30     from_u = input("Convert from (meters/kilometers/miles/feet): ").strip().lower()
31     to_u = input("Convert to (meters/kilometers/miles/feet): ").strip().lower()
32
33     try:
34         result = convert_distance(dist, from_u, to_u)
35         print(f"(dist) (from_u) is equal to (result:.2f) (to_u).")
36     except ValueError as e:
37         print(e)
```

Output :



```
(.venv) PS D:\AIASSCoding> & D:/AIASSCoding/.venv/Scripts/python.exe d:/AIASSCoding/distance.py
Enter distance: 34
Convert from (meters/kilometers/miles/feet): meters
Convert to (meters/kilometers/miles/feet): miles
34.0 meters is equal to 0.02 miles.
(.venv) PS D:\AIASSCoding>
```

STAGE 2:

Code :

```

distance.py > ...
1 # Write a Python function that converts kilometers to miles
2 # and miles to kilometers based on user input.
3 def convert_distance(value, unit):
4     """Convert distance between kilometers and miles."""
5     if unit == 'km':
6         # Convert kilometers to miles
7         return value * 0.621371
8     elif unit == 'miles':
9         # Convert miles to kilometers
10        return value / 0.621371
11    else:
12        return "Error: Invalid unit. Use 'km' or 'miles'."
13 # Example usage
14 if __name__ == "__main__":
15     user_value = float(input("Enter the distance value: "))
16     user_unit = input("Enter the unit (km or miles): ").strip().lower()
17     converted_value = convert_distance(user_value, user_unit)
18     if isinstance(converted_value, float):
19         if user_unit == 'km':
20             print(f"{user_value} kilometers is equal to {converted_value:.2f} miles.")
21         else:
22             print(f"{user_value} miles is equal to {converted_value:.2f} kilometers.")
23     else:
24         print(converted_value)

```

Output :

```

(.venv) PS D:\AIASSCoding> & D:/AIASSCoding/.venv/Scripts/python.exe d:/AIASSCoding/distance.py
Enter the distance value: 37
Enter the unit (km or miles): km
37.0 kilometers is equal to 22.99 miles.

```

STAGE 3 :

Code :

```

distance.py > ...
1 # Write a Python function convert_distance(value, unit).
2 # If unit is "km", convert kilometers to miles.
3 # If unit is "miles", convert miles to kilometers.
4 # Use correct formulas (1 km = 0.621371 miles).
5 # Handle invalid units and negative values properly.
6 def convert_distance(value, unit):
7     """Convert distance between kilometers and miles."""
8     if value < 0:
9         return "Distance cannot be negative"
10
11     if unit == "km":
12         # Convert kilometers to miles
13         miles = value * 0.621371
14         return miles
15     elif unit == "miles":
16         # Convert miles to kilometers
17         kilometers = value / 0.621371
18         return kilometers
19     else:
20         return "Invalid unit. Please use 'km' or 'miles'."
21 # example usage
22 if __name__ == "__main__":
23     # Test cases
24     print(convert_distance(10, "km")) # Expected output: 6.21371
25     print(convert_distance(6.21371, "miles")) # Expected output: 10.0
26     print(convert_distance(-5, "km")) # Expected output: "Distance cannot be negat
27     print(convert_distance(10, "meters")) # Expected output: "Invalid unit. Please use
28

```

Output :

```

(.venv) PS D:\AIASSCoding> & D:/AIASSCoding/.venv/Scripts/python.exe d:/AIASSCoding/distance.py
6.21371
10.0
Distance cannot be negative
Invalid unit. Please use 'km' or 'miles'.

```

Final Observation :

When a vague prompt was used, the AI generated unclear or very general conversion code. After specifying the type of conversion, the AI produced a basic one-way converter. When detailed instructions, formulas, and validation rules were added, the AI generated an accurate, well-structured, and reusable unit conversion function. This proves that higher prompt specificity leads to better code quality, accuracy, and reliability.